

Meta Android Developer Professional Certificate

- React Notları

Meta Android Developer Professional Certificate - React Notları

Tarih: 01.12.2025 / 13.12.2025

Konu Başlıkları: React'e Giriş, JSX, Component'ler (Fonksiyonel & Class), Props, State (useState), Event Handling, Data Flow (Parent-Child), Hooks (useState, useContext, useReducer), Conditional Rendering, Navigation (React Router), Assets (Görsel, Video)

React Nedir? Neden Kullanılır?

Meta'daki bir yazılım mühendisi (Katie) React'in Meta'da ve sektörde nasıl kullanıldığını anlatıyor. React **front-end kütüphanesi**. Okulda öğrendiğimiz kalıtım (inheritance) tabanlı OOP'den farklı olarak **composition** kullanıyor. Yani bileşenleri birleştirerek uygulama yapıyoruz.

Neden React?

- Öğrenmesi ve başlaması kolay.
- Custom UI**'lar oluşturmada çok esnek.
- Kod tekrar kullanımı (reusability) çok iyi. Aynı component'leri farklı yerlerde kullanabiliyorsun. Meta'da Facebook, Instagram, Messenger arasında bile component paylaşımı yapılıyor. Bir bug'ı core component'ta fix'lediğinde, o component'ı kullanan her yer düzeliyor.
- Angular gibi "full-fledged" bir framework değil, sadece **client-side library**. Bu yüzden routing, server-client comm gibi şeyler için başka kütüphanelerle entegre etmen gerekiyor (bu bir esneklik).
- Büyük ve aktif bir community'si var. YouTube, dökümantasyon, open source projeler... Çok kaynak var.

Single-Page Application (SPA): React ile yapılan modern uygulamalar genelde SPA. Tek bir sayfa yükleniyor, kullanıcı etkileşimine göre içerik değişiyor. URL değişmiyor (genelde). Bu, logo/navigation/footer gibi tekrar eden kısımların her seferinde yeniden yüklenmesini engelliyor, daha verimli.

Component-Based Architecture & Virtual DOM

React'in temel yapı taşı **component'lar**. Her component kendi içinde tanımlı, bağımsız, tekrar kullanılabilir bir kod parçası.

Avantajları:

- Farklı geliştiriciler aynı projede, birbirinin koduna karışmadan component'lar üzerinde çalışabilir.
- Tüm UI, basit component'ların birleşmesiyle oluşan daha karmaşık component'lardan oluşur. Örneğin bir e-ticaret checkout sayfası: Header, Payment Form, Sidebar (Order Summary) ayrı component'lar olabilir.
- Her component kendi HTML, CSS ve JS logic'ine sahip.

Virtual DOM: Eskiden DOM'u direkt manipüle etmek "spaghetti code"a yol açıyordu (karmaşık, anlaşılmaz). React, gerçek DOM'u değil, **bellekteki bir kopyası** olan Virtual DOM'u günceller. Sadece gereken minimum değişiklikleri tespit edip gerçek DOM'a uygular. Bu, performansı artırır.

Functional Components & JSX

React'te iki tür component var: **Functional** ve **Class** (eskiden daha çok kullanılırdı, şimdi functional + hooks daha yaygın).

Functional Component: Normal bir JavaScript fonksiyonu gibi. JSX return eder.

Root Component: Her React uygulamasında en az bir component olmalı, buna `App` component'ı deniyor genelde. `index.js` içinde `App` component'ı `ReactDOM.render()` ile `id="root"` olan div'e yükleniyor.

JSX (JavaScript XML): JavaScript içinde HTML benzeri yazmamızı sağlayan syntax extension. Aslında HTML değil. Dinamik içerik oluşturmak için mükemmel. JavaScript değişkenlerini `{title}` şeklinde süslü parantez içinde kullanabiliyoruz.

Önemli Kural: Component isimleri **daima büyük harfle** başlamalı (`Heading`). React, küçük harfle başlayanları HTML elementi sanıyor.

Component Oluşturma Örneği:

```
// Heading.js
function Heading() {
  const title = "Bu bir başlık metni";
  return (
    <h1>{title}</h1>
  );
}
export default Heading;
```

Transpiling: JSX, tarayıcının anlayacağı hale getirilmek için (genelde Babel ile) HTML/JavaScript'e dönüştürülür. Bu işleme transpiling denir.

React Proje Yapısı

`create-react-app` (CRA) ile proje oluşturduğunda gelen klasörler:

- **node_modules:** Tüm npm paketlerinin (bağımlılıkların) kurulu olduğu yer. Elle kurcalamaya gerek yok.
- **public:** Kullanıcıya sunulan statik asset'ler (favicon, logo, `index.html`). **En önemlisi `index.html`.** React app'imiz bu dosyanın içindeki `<div id="root"></div>` elementine enjekte oluyor.
- **src:** Tüm component'ların ve ana uygulama mantığının olduğu klasör. Biz burada çalışıyoruz.
 - `index.js`: Giriş noktası. ReactDOM.render burada.
 - `App.js`: Ana root component.
 - Diğerleri: CSS'ler, test dosyaları, vs. İstersen silebilirsin, referans veren kodları da temizlemeyi unutma.

Root'taki Dosyalar:

- `package.json`: Proje bilgileri, script'ler, bağımlılıklar listesi.
- `.gitignore`: Git'in takip etmeyeceği dosyalar.
- `README.md`: Proje açıklaması.

Modüller, Import & Export

Component'ları düzenlemek için `src` altında `components` klasörü oluşturup component dosyalarını oraya koymak yaygın bir uygulama.

Export (Dışa Aktarma): Bir component'ı başka bir dosyada kullanılabilir yapar.

- `export default Heading`: **Default export.** Import ederken istediğimiz ismi verebiliriz. Dosyada genelde bir tane olur.
- `export { Heading, Footer }`: **Named export.** Import ederken aynı isimle çekmek gerekir.

Import (İçe Aktarma): Başka bir dosyadan component/modül almak.

```
import Heading from './components/Heading'; // Default import
import { Footer } from './components/Footer'; // Named import
```

Props (Properties)

Component'lar arasında veri aktarmak için kullanılır. JavaScript objesi gibi. **Parent component'tan Child component'a** tek yönlü bir akış vardır.

Parent Component (Veriyi Gönderen):

```
function App() {
  return (
```

```
    <Header name="Anna" color="purple" />
  );
}
```

Child Component (Veriyi Alan):

```
function Header(props) {
  // props.name -> "Anna"
  // props.color -> "purple"
  return <h1 style={{color: props.color}}>Merhaba, {props.name}</h1>;
}
```

Kısa yol: `function Header({ name, color })` şeklinde destructuring de yapabiliriz.

Önemli: Props **read-only**'dir (salt okunur). Child component, gelen props'u değiştiremez. Ayrıca, props sadece parent'tan child'a gider, tam tersi yönde props ile veri akışı olmaz (bunun için state/farklı yöntemler var).

JSX Kuralları & Styling

- JSX return ederken, birden fazla satır ise **parantez** `()` içine al.
- Tüm JSX elementleri **tek bir parent element** içinde olmalı. Eskiden `div` kullanılırdı, şimdi `<> </>` (**Fragment**) kullanmak daha temiz.
- CSS class vermek için `class` değil, `className` kullan. (Çünkü `class` JavaScript'te reserved keyword).
- Inline style verirken de **camelCase** kullan: `style={{ backgroundColor: 'blue', fontSize: '20px' }}`
- JavaScript ifadelerini `{ }` içine yaz.

Embedded Expressions (Gömülü İfadeler): JSX içine JavaScript değişkeni, fonksiyon çağırısı vs. koyabiliriz.

```
function Logo({ user }) {
  const profilePic = <img src={user.imageUrl} />;
  return <div>{profilePic}</div>;
}
```

Event Handling (Olay İşleme)

HTML'de `onclick` gibi event'ler React JSX'te **camelCase** yazılır: `onClick`.

Event Handler Nasıl Eklenir?

```
function Btn() {  
  const handleClick = () => {  
    console.log("Tıklandı!");  
  };  
  
  return (  
    <button onClick={handleClick}>Tıkla Bana</button>  
  );  
}
```

Dikkat: `onClick={handleClick()}` şeklinde **fonksiyonu çağırarak** verme! Sadece referansını ver (`handleClick`). Yoksa component render olur olmaz fonksiyon çalışır, tıklamayı beklemez.

Farklı Event'ler: `onMouseOver`, `onCopy`, `onSubmit` gibi bir sürü event var.

Event ile State Değiştirme Örneği (Toggle):

```
function ModeToggleler() {  
  const [darkModeOn, setDarkModeOn] = useState(true);  
  const darkMode = <h1>Dark Mode Açık</h1>;  
  const lightMode = <h1>Light Mode Açık</h1>;  
  
  const handleClick = () => {  
    setDarkModeOn(!darkModeOn); // state'i tersine çevir  
    console.log(darkModeOn ? "Dark Mode Açık" : "Light Mode Açık");  
  };  
  
  return (  
    <div>  
      {darkModeOn ? darkMode : lightMode}  
      <button onClick={handleClick}>Modu Değiştir</button>  
    </div>  
  );  
}
```

State (Durum) & `useState` Hook

Component'in **içsel, değişebilir verisi**. Props'tan farkı, component'in kendi kontrolünde olması. State değiştiğinde, component otomatik olarak yeniden render olur (ekranda güncellenir).

`useState` Hook: State oluşturmak ve yönetmek için kullanılır.

```
import { useState } from 'react';
```

```
function App() {  
  // Destructuring yapıyoruz:  
  // `word` -> state değişkeninin şu anki değeri  
  // `setWord` -> state'i güncelleyecek fonksiyon  
  // `useState("Eat")` -> state'in başlangıç değeri "Eat"  
  const [word, setWord] = useState("Eat");  
  
  const handleClick = () => {  
    setWord("Drink"); // State'i "Drink" olarak güncelle  
  };  
  
  return (  
    <div>  
      <h1>{word} at Little Lemon</h1>  
      <button onClick={handleClick}>Tıkla</button>  
    </div>  
  );  
}
```

Kural: State'i asla direkt değiştirme (`word = "Drink"` YANLIŞ!). Her zaman `setWord("Drink")` gibi **setter fonksiyonu** kullan.

State vs Props:

- **Props:** Dışarıdan gelir, değiştirilemez (immutable). Parent -> Child akar.
- **State:** İçeridedir, component kendi değiştirir (mutable). Değişince component yeniden render olur.

State Management (State Yönetimi) & Context API

Uygulama büyüdükçe state'i component'lar arasında taşımak zorlaşabilir. Özellikle **prop drilling** (state'i birçok component katmanından aşağı geçirme) sorun yaratır.

Çözüm Yolları:

1. **Lifting State Up:** State'i, ihtiyaç duyan tüm component'ların ulaşabileceği en yakın ortak parent'a taşımak.
2. **Context API:** State'i global olarak yönetmek. Prop drilling'e gerek kalmaz.

Context API Temel Kullanım:

- `React.createContext()` ile bir context oluştur.
- `<MyContext.Provider value={state}>` ile state'i sağla.
- İhtiyacı olan child component'lar `useContext(MyContext)` hook'u ile state'e direkt erişir.

`useReducer` **Hook:** Daha karmaşık state mantığı için kullanılır. `useState`'e benzer ama state güncellemeleri bir `reducer` fonksiyonu tarafından yönetilir (Redux pattern'ine benzer).

Stateful vs Stateless Components

- **Stateful Component:** Kendi iç state'ine sahip component. Genellikle **smart component** olarak da adlandırılır. Veriyi yönetir, state'i değiştirir, child component'lara props olarak geçirir.
- **Stateless Component:** Kendi state'i yoktur. Sadece parent'tan gelen props'u alır ve görüntüler. Genellikle **presentational/dumb component** olarak adlandırılır. Sadece UI render etmekle ilgilenir.

Genel Pattern: State'leri birkaç stateful (akıllı) component'ta topla, veriyi stateless (görsel) component'lara props olarak dağıt.

Navigation (React Router)

React tek sayfa uygulama (SPA) yapısında olduğu için sayfalar arası geçiş normal `<a>` tag'i ile olmaz (sayfa yenilenir). Bunun için **React Router** kütüphanesi kullanılır.

Kurulum: `npm install react-router-dom@6`

Temel Kullanım:

```
// index.js'de
import { BrowserRouter } from 'react-router-dom';
// App component'ını BrowserRouter ile sarmala
<BrowserRouter>
  <App />
</BrowserRouter>

// App.js'de
import { Routes, Route, Link } from 'react-router-dom';

function App() {
  return (
    <div>
      <nav>
        {/* Normal <a> değil, <Link> kullan */}
        <Link to="/">Anasayfa</Link>
        <Link to="/about">Hakkında</Link>
      </nav>
      <Routes>
        {/* Hangi URL'de hangi component render edilsin? */}
        <Route path="/" element={<Homepage />} />
      </Routes>
    </div>
  );
}
```

```
    <Route path="/about" element={<AboutMe />} />
  </Routes>
</div>
);
}
```

Navbar, Sidebar, Burger Menu gibi yapılar stil ile şekillendirilir, mantık React Router ile yönetilir.

Conditional Rendering (Koşullu Render)

JSX içinde bir koşula göre farklı içerik göstermek.

Yöntem 1: Ternary Operator (En yaygın)

```
{isLoggedIn ? <AdminPanel /> : <LoginForm />}
```

Yöntem 2: Logical && Operator (Sadece true ise göster)

```
{unreadMessages.length > 0 && <h2>Okunmamış mesajlarınız var.</h2>}
```

Yöntem 3: If-Else (Component içinde, return'den önce)

```
function CurrentMessage() {
  const day = new Date().getDay();
  let dayMessage;

  if (day >= 1 && day <= 5) {
    dayMessage = <Workdays />;
  } else {
    dayMessage = <Weekends />;
  }

  return <div>{dayMessage}</div>;
}
```

Assets (Görseller, Video, Ses)

Asset Nedir: Uygulamanın çalışması için gerekli harici dosyalar (resim, font, video, CSS).

Nereye Koymalı?

- **public** klasörü: Favicon gibi component'ların doğrudan bağlı olmadığı, sabit dosyalar.
- **src/assets** klasörü (önerilen): Component'ların içe aktarıp kullandığı dosyalar.

Görsel Ekleme:


```
// Yöntem 1: import
import catImage from './assets/images/cat.jpg';
function Comp() { return <img src={catImage} alt="Kedi" />; }

// Yöntem 2: require (artık pek kullanılmıyor)
function Comp() { return <img src={require('./assets/images/cat.jpg')} alt="Kedi" />; }

// Yöntem 3: URL direkt
function Comp() { return ; }
```

Video/Audio Ekleme:

- Yerel dosya için HTML5 `<video>` veya `<audio>` tag'ini kullanabilirsin.
- YouTube, Vimeo gibi yerlerden embed etmek için **React Player** gibi hazır paketler kullanmak çok daha kolay.

React Player Kullanımı:

```
npm install react-player
```

```
import ReactPlayer from 'react-player';

function VideoComp() {
  return (
    <ReactPlayer
      url="https://youtube.com/watch?v=xxx"
      controls={true}
      playing={false} // otomatik oynatma kapalı
      volume={0.5} // %50 sesle başla
    />
  );
}
```